

1. What is the role of an Error Boundary in React?

An **Error Boundary** is a React component that acts as a JavaScript catch block for the declarative UI (React components). Its primary role is to **catch JavaScript errors anywhere in its child component tree, log those errors, and display a fallback UI** instead of crashing the entire application.

Introduced in React 16, Error Boundaries prevent the entire application from unmounting due to an uncaught error during rendering, in lifecycle methods, or in constructors of the tree below them.

Key characteristics and roles:

- **Graceful Degradation:** They allow the application to display a friendly error message (e.g., "Something went wrong") while keeping the rest of the application functional.
- **Isolation:** They isolate the error to a specific part of the UI. If a sidebar component fails, the main content can still remain interactive.
- **Implementation:** A component becomes an Error Boundary by defining either (or both) of the lifecycle methods:
 - `static getDerivedStateFromError(error):` Used to render a fallback UI after an error is thrown.
 - `componentDidCatch(error, info):` Used to log error information (e.g., sending to an error reporting service like Sentry).

What Error Boundaries do NOT catch:

- Event handlers (errors inside event handlers are handled with regular `try...catch`).
- Asynchronous code (e.g., `setTimeout` or `requestAnimationFrame` callbacks).
- Server-side rendering.
- Errors thrown in the Error Boundary itself (rather than its children).

2. How does the try...catch block differ from an Error Boundary in handling errors in React?

While both `try...catch` and Error Boundaries handle JavaScript errors, they operate at fundamentally different levels of abstraction and scope. `try...catch` is an imperative JavaScript construct, whereas Error Boundaries are a declarative React structure.

Feature	try...catch Block	Error Boundary
Paradigm	Imperative	Declarative (React-specific)
Scope	Handles errors within a specific block of imperative code (e.g., inside a function, event handler, or specific lines of logic).	Handles errors for an entire subtree of React components. It wraps declarative JSX markup.
Use Case	Event Handlers, Async Code, & Logic: Ideal for	Rendering Logic & Lifecycles: Ideal for catching errors during the render phase , in constructors, and in lifecycle methods

Feature	try...catch Block	Error Boundary
	catching errors during user interactions (e.g., onClick), API calls (useEffect), or complex utility functions.	(e.g., componentDidMount).
Syntax	Wraps logic in a block: try { ... } catch (err) { ... }	Declares a class component with specific methods: static getDerivedStateFromError and componentDidCatch.
UI Preservation	Cannot preserve the React component tree on its own. If a render error occurs inside a try...catch, you must manually manage state to decide what UI to show.	Automatically prevents unmounting of the whole component tree. It preserves the parent UI and replaces the broken subtree with a fallback UI without manual state management in the parent.
Granularity	Low-level. You place it exactly where the potential error occurs.	High-level. You place it around a section of your app (e.g., <ErrorBoundary><Widget /></ErrorBoundary>) to protect that entire feature.

Summary of Differences

When to use try...catch:

You use try...catch inside **event handlers** or **asynchronous functions** because Error Boundaries cannot catch these errors.

javascript

```
const handleClick = () => {
  try {
    riskyOperation();
  } catch (error) {
    console.error("Handled in try/catch");
    setErrorState(true);
  }
};
```

When to use Error Boundaries:

You use Error Boundaries to wrap components to catch errors that occur during **rendering** or in **lifecycle methods**.

javascript

```
<ErrorBoundary fallback={<h1>Widget failed to load</h1>}>
  <MyCriticalComponent />
</ErrorBoundary>
```

In essence, try...catch is for protecting specific lines of imperative logic, while Error Boundaries are for protecting the declarative UI structure (the component tree) from rendering errors.

Note: As per your instruction, the content above is formatted for an assignment. To submit, please copy this content into a text editor (like Microsoft Word or Google Docs) and export/save the file as a **PDF**.

